WEEK 9

Math for ML II

Probability, Distributions, and Core ML Functions

Day 41	**Probability Basics**	Events, outcomes, empirical vs theoretical, Monte Carlo
Day 42	**Distributions**	Uniform, normal, mean/variance, skewness, multimodality
Day 43	**Computing Probabilities**	PDF, CDF, ppf, SciPy distributions, tail probabilities
Day 44	**Log, Exp, Softmax**	Numerical stability, log probabilities, temperature scaling
Day 45	**Math Mini-Project**	Integrating LA + probability in classification task

PROBABILITY AND SOFTMAX IN ML

```
import numpy as np; from scipy import stats
prob_correct = np.mean(predictions == y_true)  # Empirical probability
probs = softmax(scores)                        # Scores → probabilities
loss = -np.log(probs[true_class])              # Cross-entropy loss
```

DAY 41

Probability Basics

Uncertainty, Outcomes, and ML Predictions

OBJECTIVES

- Build intuition for probability as long-run frequency
- Understand events, outcomes, and sample spaces
- Distinguish empirical vs theoretical probability
- Connect probability to ML predictions
- Use seeds for reproducibility

ACTIVITY

Simulate coin flips and dice rolls.

Estimate probabilities empirically.

ASSESSMENT

Quiz on probability definitions + code snippets.

What Probability Represents

Fraction of times an event would occur over many trials

Intuitive Meaning

- P(event) ≈ times event occurs / total trials
- Values range 0 (never) to 1 (always)
- Think: "If I repeated this forever, how often?"

In ML Systems

- Predicted probabilities reflect uncertainty about labels
- Accuracy = empirical probability of correct prediction
- Error rate = probability of making a mistake

EMPIRICAL PROBABILITY

```
def empirical_probability(num_trials=10000):
    count = sum(1 for _ in range(num_trials) if random.random() < 0.3)
    return count / num_trials  # Converges to 0.3 as trials → ∞
print("P(x < 0.3) ≈", empirical_probability())
```

Outcomes, Events, and Sample Space

Formalizing randomness in code

Concepts

- Outcome: single result (e.g., "Heads", "3")
- Sample space: set of all possible outcomes
- Event: subset of outcomes we care about

ML Connection

- ML "events": model predicts correctly, loss < threshold
- Evaluation metrics = probabilities of events
- Understanding events helps interpret model behavior

SAMPLE SPACE AND EVENTS

```
coin_space = ["H", "T"];
die_space = [1, 2, 3, 4, 5, 6]
event_even = {x for x in die_space if x % 2 == 0}  # {2, 4, 6}
event_high = {x for x in die_space if x >= 4}     # {4, 5, 6}
print("P(even) =", len(event_even)/len(die_space))  # 0.5
```

Empirical vs Theoretical Probability

Simulation approximates ideal probability

Theoretical

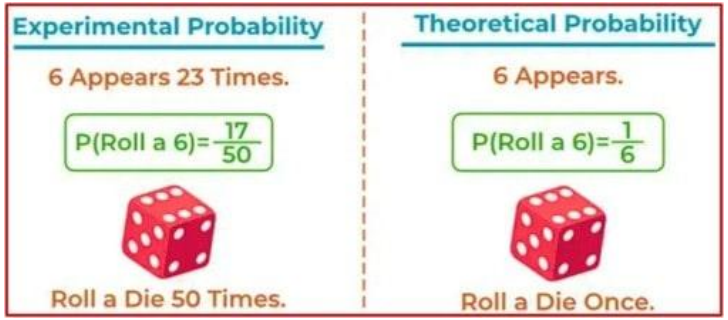
- P(Heads) = 0.5 for ideal fair coin
- Exact, derived from assumptions
- Doesn't need data to compute

Empirical

- Estimated from repeated trials
- Fluctuates but converges to theoretical
- Test set metrics are empirical estimates!

COIN FLIP SIMULATION

```
def simulate_coin_flips(n=5000):
    outcomes = np.random.choice(["H", "T"], size=n, p=[0.5, 0.5])
    return np.mean(outcomes == "H")  # Empirical P(Heads)
p_heads = simulate_coin_flips()  # ≈ 0.5, but rarely exactly 0.5
```



Randomness vs Determinism — Using Seeds

Making random code reproducible

Why Seeds Matter

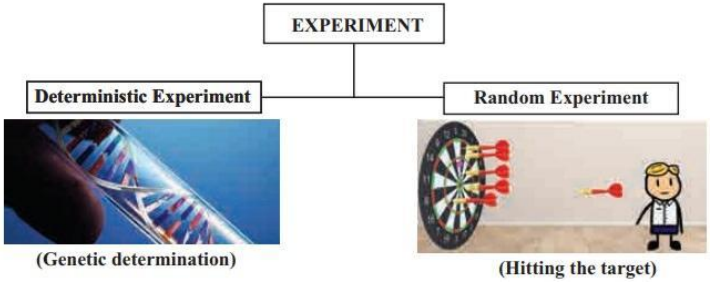
- Deterministic: same inputs → same outputs
- Random: includes shuffling, initialization
- Seeds make random operations repeatable

ML Relevance

- Reproducibility critical for experiments
- Random weight init affects training trajectory
- Without seeds, debugging becomes harder

USING SEEDS FOR REPRODUCIBILITY

```
np.random.seed(41)
run1 = np.random.randint(0, 10, size=5)  # [7, 0, 1, 8, 2]
np.random.seed(41)
run2 = np.random.randint(0, 10, size=5)  # Identical! [7, 0, 1, 8, 2]
```



Probability in ML Predictions

Thresholding probabilities into decisions

Binary Classification

- Model outputs P(class=1) for each example
- Threshold (e.g., 0.5) converts to hard label
- Probabilities = uncertainty; threshold = decision

Trade-offs

- Higher threshold → fewer false positives
- Lower threshold → fewer false negatives
- Choose based on application costs!

THRESHOLDING PREDICTIONS

```
predicted_probs = np.array([0.1, 0.4, 0.6, 0.8])
threshold = 0.5
predicted_labels = (predicted_probs >= threshold).astype(int)
print("Probs:", predicted_probs, "→ Labels:", predicted_labels)
```

Simulating a Die — Event Probabilities

Monte Carlo estimation of discrete events

Goal

- Estimate P(even) and P(roll >= 4) empirically
- True values: P(even) = P(>=4) = 0.5
- Simulation should converge near 0.5

ML Connection

- Models assume data from some distribution
- Simulation tests assumptions and robustness
- Monte Carlo central to many ML algorithms

DIE ROLL SIMULATION

```
def simulate_die(n=10000):
    rolls = np.random.randint(1, 7, size=n)  # 1 to 6
    return np.mean(rolls % 2 == 0), np.mean(rolls >= 4)
p_even, p_ge4 = simulate_die()  # Both ≈ 0.5
```

Monte Carlo Intuition

Probability as long-run frequency via sampling

The Idea

- Use random sampling to approximate probabilities
- Replaces tedious calculations with simulation
- More samples → better approximation

ML Relevance

- Stochastic gradient descent is Monte Carlo
- Sampling-based inference (MCMC, variational)
- Uncertainty estimation via sampling

MONTE CARLO: P(sum of 2 dice >= 10)

```
def prob_sum_ge10(n=100000):
    die1 = np.random.randint(1, 7, n); die2 = np.random.randint(1, 7, n)
    return np.mean(die1 + die2 >= 10)  # ≈ 1/6 = 0.167
print("P(sum >= 10):", prob_sum_ge10())
```

Conditional Probability via Simulation

P(A|B) — probability of A given B occurred

Concept

- P(A|B) = P(A and B) / P(B)
- Restrict to trials where B happened
- Then count how often A also occurred

ML Examples

- P(churn | segment="A")
- P(error | high_confidence)
- Many ML questions are conditional!

P(EVEN | ROLL >= 4)

```
rolls = np.random.randint(1, 7, 100000)
event_B = rolls >= 4  # B = roll is 4, 5, or 6
event_A_and_B = (rolls % 2 == 0) & event_B
p_even_given_ge4 = event_A_and_B.sum() / event_B.sum()  # ≈ 2/3
```

Independence vs Dependence

When knowing B tells you nothing about A

Independence

- P(A|B) = P(A) — B doesn't affect A
- Independent events: P(A and B) = P(A)×P(B)
- Many models assume feature independence

Dependence Impact

- Strong dependence breaks naive assumptions
- May need more expressive models
- Correlated features = dependent!

INDEPENDENT VS DEPENDENT

```
A_ind = np.random.binomial(1, 0.5, 100000)
B_ind = np.random.binomial(1, 0.5, 100000)  # Independent of A
B_dep = A_ind.copy(); B_dep[np.random.rand(100000) < 0.1] ^= 1  # Dependent
print("P(A==B) indep:", np.mean(A_ind==B_ind))  # ≈ 0.5
print("P(A==B) dep:", np.mean(A_ind==B_dep))    # ≈ 0.9
```

DAY 41 · ACTIVITY

Probability Simulation Practice

Estimate probabilities via Monte Carlo

Activity Steps

1. Choose coin or die experiment
2. Define at least 2 events to estimate
3. Simulate ≥ 10,000 trials with np.random
4. Compute empirical probabilities
5. Compare to theoretical values
6. Note convergence behavior

Assessment Questions

Q1: What does P = 0.2 mean intuitively?

Q2: Empirical vs theoretical probability?

Q3: What is a sample space and event?

Q4: Why fix random seeds in ML?

Q5: Probabilities in binary classification?

DAY 42

Distributions

How Probability is Spread Over Values

OBJECTIVES

- Understand distributions as probability spread
- Build intuition for uniform and normal
- Relate mean, variance, spread to data behavior
- See why normality assumptions appear in ML
- Recognize skewness and multimodality

ACTIVITY

Generate and visualize samples from uniform and normal distributions.

ASSESSMENT

Submit normal distribution plot + interpretation.

What a Distribution Represents

How likely different values are

Intuitive Meaning

- Describes probability for each possible value
- Discrete: probability for each value
- Continuous: relative density over intervals

ML Connection

- Data distributions describe features/targets
- Model assumptions (e.g., Gaussian noise)
- Guides transformations, models, loss functions

DISCRETE DISTRIBUTION FROM SIMULATION

```
values, counts = np.unique(np.random.randint(1, 7, 1000), return_counts=True)
probs = counts / counts.sum()  # Probability per outcome
print("Die outcomes:", values, "\nEstimated probs:", probs)
```

Discrete vs Continuous Distributions

Integers vs real numbers

Discrete

- Takes integer values (counts)
- Examples: clicks, purchases, dice rolls
- Use Poisson, Binomial, etc.

Continuous

- Takes real values
- Examples: heights, noise, scores
- Use Normal, Exponential, etc.

DISCRETE VS CONTINUOUS SAMPLES

```
discrete = np.random.poisson(lam=3.0, size=1000)     # Counts
continuous = np.random.normal(loc=0, scale=1, size=1000)  # Real
print("Discrete first 10:", discrete[:10])
print("Continuous first 10:", continuous[:10].round(2))
```

Uniform Distribution

All values equally likely in range

Properties

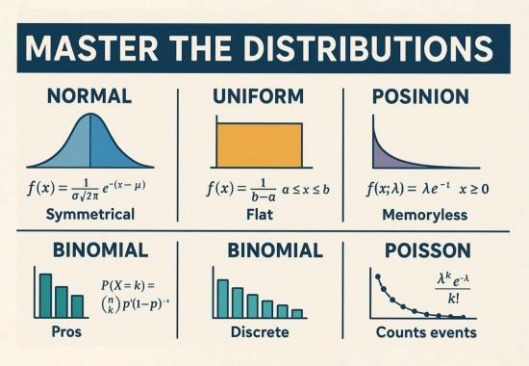
- Flat density across [low, high]
- No preferred value in range
- Mean = (low + high) / 2

ML Uses

- Weight initialization: np.random.uniform(-1, 1)
- Random train/val splits (shuffled)
- No prior reason to prefer any value

UNIFORM DISTRIBUTION

```
uniform = np.random.uniform(-1, 1, size=5000)
print("Mean:", uniform.mean(), "Std:", uniform.std())
# Histogram shows flat bars across [-1, 1]
sns.histplot(uniform, bins=30, stat="density")
```



DAY 42 · DISTRIBUTIONS

Normal (Gaussian) Distribution

Bell-shaped curve centered at mean

Properties

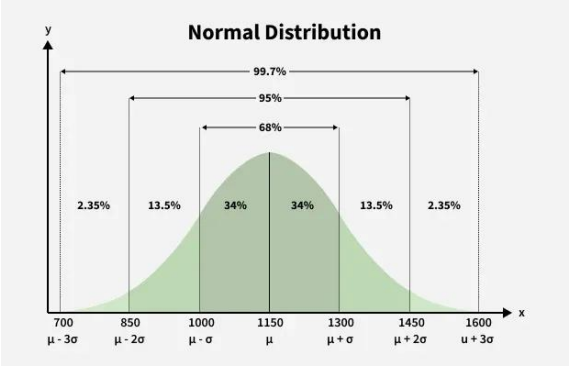
- Symmetric around mean μ
- Spread controlled by std dev σ
- ~68% within ±1σ, ~95% within ±2σ

Why So Common in ML

- Central Limit Theorem: sums → normal
- Gaussian noise → tractable loss functions
- Many models assume normal errors

NORMAL DISTRIBUTION

```
normal = np.random.normal(loc=0, scale=1, size=5000)
print("Mean:", normal.mean(), "Std:", normal.std())  # ≈ 0, ≈ 1
sns.histplot(normal, bins=40, stat="density", kde=True)  # Bell shape
```



Mean, Variance, and Spread

Center and spread of a distribution

Mean

- Average value = center of distribution
- np.mean(data) estimates μ
- Tells "typical" value

Variance / Std Dev

- Average squared deviation from mean
- Larger variance = more spread
- std = sqrt(variance)

COMPUTING MEAN AND VARIANCE

```
data = np.random.normal(loc=5.0, scale=2.0, size=10000)
print("Mean:", np.mean(data))  # ≈ 5.0
print("Variance:", np.var(data), "Std:", np.std(data))  # ≈ 4.0, ≈ 2.0
```

(Mean) $\mu = \frac{\sum_{i=1}^{N} x_i}{N}$

(Variance) $\sigma^2 = \frac{\sum_{i=1}^{N} (x_i - \mu)^2}{N}$

(Standard Deviation) $\sigma = \sqrt{\frac{\sum_{i=1}^{N} (x_i - \mu)^2}{N}}$

Visualizing Different Normal Distributions

How μ and σ change the shape

Changing Mean

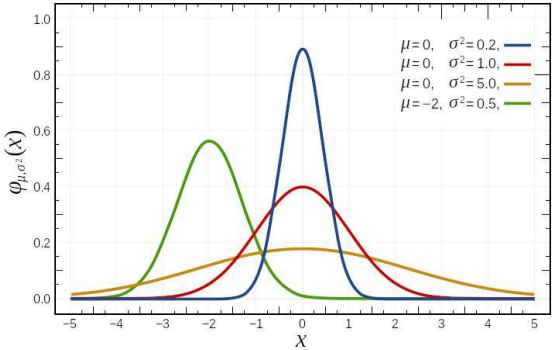
- Shifts distribution left or right
- Peak moves to new μ
- Spread stays the same

Changing Std Dev

- Larger σ → wider, flatter bell
- Smaller σ → narrower, taller bell
- Feature scaling standardizes these!

COMPARING DISTRIBUTIONS

```
x1 = np.random.normal(0, 1, 5000)   # μ=0, σ=1
x2 = np.random.normal(2, 0.5, 5000) # μ=2, σ=0.5 (narrower, shifted)
sns.kdeplot(x1, label="N(0,1)"); sns.kdeplot(x2, label="N(2,0.5)")
```



Why Normality Assumptions Appear in ML

Central Limit Theorem and tractability

CLT

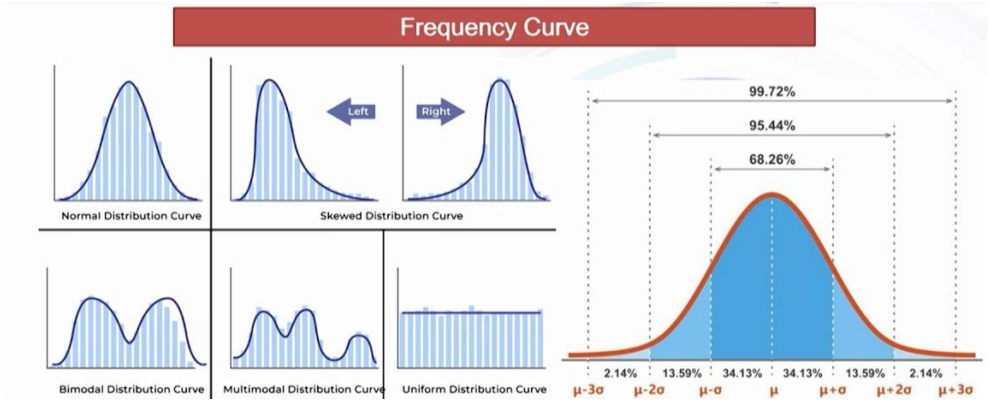
- Sum of many independent factors → normal
- Even non-normal components sum to normal
- Justifies assuming Gaussian errors

Cautions

- Not all data is normal!
- Heavy tails break assumptions
- Always check distribution before assuming

CLT DEMONSTRATION

```
uniforms = np.random.uniform(0, 1, (10000, 10))
sums = uniforms.sum(axis=1)  # Sum of 10 uniforms per trial
# sums looks approximately normal despite uniform components!
```



Skewed and Heavy-Tailed Distributions

When normal assumptions fail

Skewness

- Right-skewed: long tail to the right
- Many small values, few very large
- Mean > median for right-skewed

Heavy Tails

- Extreme values more common than normal
- Outliers can destabilize training
- Log transform or robust losses help

LOGNORMAL (SKEWED)

```
lognormal = np.random.lognormal(mean=0, sigma=1, size=5000)
print("Mean:", lognormal.mean(), "Median:", np.median(lognormal))
# Mean > Median indicates right skew; consider log transform
```

Mixture Distributions and Multimodality

Multiple peaks indicate subgroups

Multimodality

- Data from multiple subpopulations
- Multiple peaks in histogram
- Single model may be insufficient

ML Solutions

- Mixture models (GMM)
- Segment-specific models
- Clustering before modeling

BIMODAL MIXTURE

```
comp1 = np.random.normal(-2, 0.5, 3000)
comp2 = np.random.normal(2, 0.5, 3000)
mixture = np.concatenate([comp1, comp2])  # Two clear peaks!
```

DAY 42 · ACTIVITY

Distribution Visualization

Generate and plot multiple distributions

Activity Steps

1. Generate Uniform(0, 1) samples
2. Generate Normal(0, 1) and Normal(3, 2) samples
3. Optional: Poisson samples for counts
4. Plot histograms with stat="density"
5. Compute mean and std for each
6. Describe shape differences

Assessment Questions

Q1: How does variance affect normal shape?

Q2: Why is assuming normality risky?

Q3: What ML methods are distribution-sensitive?

Submit: Normal plot + interpretation bullets.

DAY 43

Computing Probabilities

PDFs, CDFs, and SciPy

OBJECTIVES

- Understand random variables as numeric outcomes
- Use .pdf(), .cdf(), .ppf() in SciPy
- Compute probabilities for intervals
- Recognize why PDF alone isn't probability
- Validate with Monte Carlo

ACTIVITY

Compute probabilities of intervals for normal distributions.

ASSESSMENT

Code submission with probability calculations.

Random Variables and Distributions

Numeric outcomes of random experiments

Concept

- Maps randomness to numbers
- Examples: # clicks, height, prediction error
- Distribution describes probability over values

In ML

- Loss values are random variables
- Gradients fluctuate across batches
- Understanding distributions aids diagnostics

MODEL RESIDUALS AS RANDOM VARIABLE

```
errors = np.random.normal(loc=0, scale=1, size=1000)  # Residuals
print("Mean residual:", errors.mean())  # Should be near 0
# Distribution of residuals informs model fit quality
```

PDF vs CDF — Core Concepts

Density vs cumulative probability

PDF

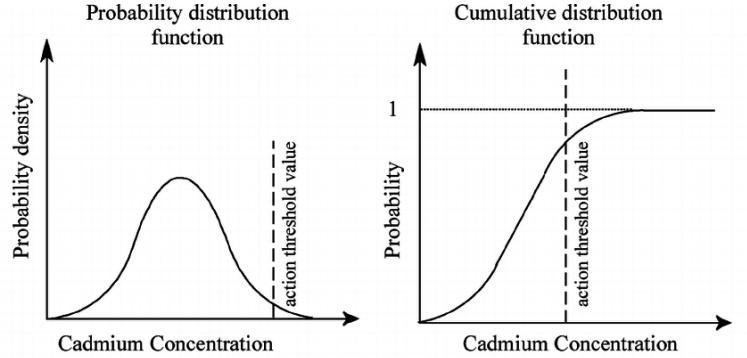
- Probability Density Function
- Height of curve at x (not probability!)
- Must integrate to get actual probability

CDF

- Cumulative Distribution Function
- $P(X \leq x)$ directly
- Useful for computing range probabilities

PDF VS CDF WITH SCIPY

```
from scipy import stats
dist = stats.norm(loc=0, scale=1)
print("PDF at x=1:", dist.pdf(1))   # ≈ 0.24 (density, not prob!)
print("CDF at x=1:", dist.cdf(1))   # ≈ 0.84 = P(X ≤ 1)
```



Probability of a Range Using CDF

$P(a \le X \le b) = CDF(b) - CDF(a)$

Formula

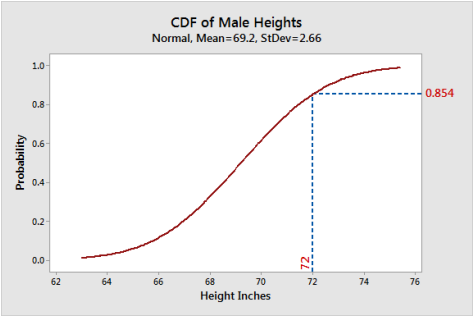
- $P(a \le X \le b) = CDF(b) - CDF(a)$
- Works for any continuous distribution
- For Normal(0,1): ~68% in ±1σ, ~95% in ±2σ

ML Use

- Confidence intervals around predictions
- Understanding uncertainty bands
- Risk assessment for extreme values

RANGE PROBABILITY

```
dist = stats.norm(0, 1)
prob_between = dist.cdf(1) - dist.cdf(-1)  # P(-1 ≤ X ≤ 1)
print("P(-1 ≤ X ≤ 1):", prob_between)  # ≈ 0.68
```



Using .ppf() — Quantiles

Inverse CDF: given p, find x

PPF Concept

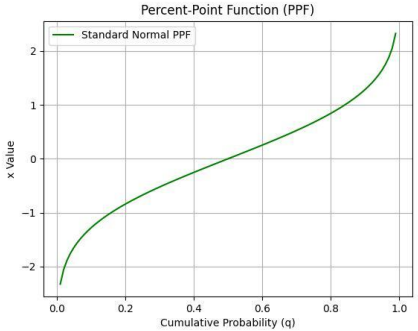
- Percent Point Function = inverse CDF
- Given probability p, return x where P(X≤x)=p
- a.k.a. quantile function

ML Applications

- 95th percentile for anomaly detection
- Prediction intervals
- Risk thresholds

QUANTILES WITH PPF

```
dist = stats.norm(0, 1)
q_05 = dist.ppf(0.05)  # 5th percentile ≈ -1.64
q_95 = dist.ppf(0.95)  # 95th percentile ≈ 1.64
print("90% of values lie between", q_05, "and", q_95)
```



Discrete Probabilities with scipy.stats

PMF for exact values

PMF vs PDF

- PMF: Probability Mass Function (discrete)
- Gives P(X = k) directly for integers
- CDF still accumulates: P(X ≤ k)

Example: Binomial

- n trials, each with success prob p
- Counts successes (like # correct predictions)
- binom.pmf(k, n, p) gives P(exactly k)

BINOMIAL PROBABILITIES

```
binom = stats.binom(n=10, p=0.3)
print("P(X=3):", binom.pmf(3))    # P(exactly 3 successes)
print("P(X≤3):", binom.cdf(3))   # P(at most 3 successes)
```

Monte Carlo Validation

Check analytical probability with simulation

Why Validate

- Catch computation errors
- Build intuition for distributions
- Empirical should match theoretical

Approach

- Sample many values from distribution
- Compute empirical probability
- Compare to CDF-based calculation

MONTE CARLO CHECK

```
dist = stats.norm(0, 1)
prob_scipy = dist.cdf(1) - dist.cdf(-1)  # Analytical
samples = dist.rvs(100000)
prob_mc = np.mean((-1 <= samples) & (samples <= 1))  # Empirical
print("Analytical:", prob_scipy, "MC:", prob_mc)  # Should match
```

Expected Value — Center of Distribution

Long-run average of random variable

Definition

- E[X] = long-run average
- Center of distribution in probability terms
- dist.mean() gives theoretical value

ML Connection

- E[loss] is what optimization minimizes
- Mini-batch loss estimates E[loss]
- Convergence: empirical → theoretical

EMPIRICAL VS THEORETICAL MEAN

```
dist = stats.expon(scale=2.0)  # Exponential with mean=2
samples = dist.rvs(100000)
print("Empirical mean:", samples.mean())  # ≈ 2.0
print("Theoretical mean:", dist.mean())  # Exactly 2.0
```

Tail Probabilities and Risk

Probability of extreme events

Tail Probability

- P(X > threshold) for rare events
- Computed as 1 - CDF(threshold)
- Quantifies risk of extreme outcomes

ML Relevance

- Worst-case errors, extreme losses
- Rare but severe model failures
- Important for risk-sensitive applications

TAIL PROBABILITY

```
dist = stats.norm(0, 1)
threshold = 2.0
prob_tail = 1 - dist.cdf(threshold)  # P(X > 2)
print("P(X > 2):", prob_tail)  # ≈ 0.023 (about 2.3%)
```

DAY 43 · ACTIVITY

Probability Calculations

Use SciPy for distribution computations

Activity Steps

1. Choose a distribution (normal, exponential, t)
2. Compute $P(X \leq a)$ using cdf
3. Compute $P(a \leq X \leq b)$ using cdf difference
4. Find x where $P(X \leq x) = 0.9$ using ppf
5. Validate one probability with Monte Carlo
6. Explain what each result means

Assessment Questions

Q1: Why isn't PDF alone sufficient for P?

Q2: Compute $P(X \geq 2)$ for Normal(0,1)?

Q3: What does ppf(0.975) represent?

Q4: Verify probability with Monte Carlo?

Q5: When are tail probabilities important?

DAY 44

Log, Exp, Softmax

Core ML Functions for Stability

OBJECTIVES

- Understand why log and exp appear in ML
- Work with log probabilities for stability
- Implement softmax and interpret outputs
- Learn numerically stable implementations
- Apply temperature scaling

ACTIVITY

Implement softmax variants and apply to score vectors.

ASSESSMENT

Code challenge: numerically stable softmax.

Exponential Function (exp)

Maps reals to positive reals

Properties

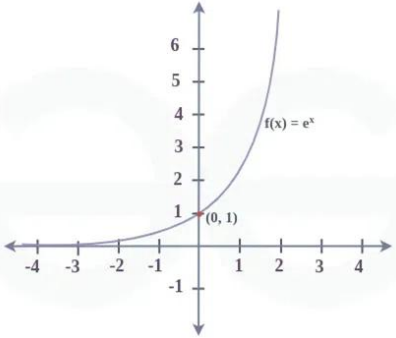
- exp(x) always positive
- Grows very quickly for positive x
- Turns additive changes into multiplicative

ML Uses

- Softmax: exp(scores) → positive weights
- Probability models (exponential family)
- Can overflow if scores too large!

EXP BEHAVIOR

```
xs = np.array([-3, -1, 0, 1, 3])
exp_xs = np.exp(xs)
print("x:", xs, "\nexp(x):", exp_xs)  # [0.05, 0.37, 1.0, 2.72, 20.1]
```



Logarithm (log)

Compresses scale, turns products into sums

Properties

- Inverse of exp: log(exp(x)) = x
- Turns products into sums: log(ab) = log(a)+log(b)
- Defined only for positive inputs

ML Uses

- Log-likelihood, log-loss, cross-entropy
- Avoids underflow when multiplying small probs
- Loss = sum of log probabilities

LOG PROBABILITIES

```
probs = np.array([0.9, 0.1, 1e-5])
log_probs = np.log(probs)
print("Probs:", probs, "\nLog probs:", log_probs)  # [-0.1, -2.3, -11.5]
```

Underflow Problem

Multiplying many small probabilities → 0

The Problem

- Multiplying 50 probabilities of 0.99 each
- Product underflows to 0 in float64
- Loses all information about likelihood

The Solution

- Work in log space: sum logs instead
- $\log(\prod p_i) = \sum \log(p_i)$
- Stable and numerically safe

LOG-SUM AVOIDS UNDERFLOW

```
probs = np.full(50, 0.99)
product = np.prod(probs)  # May underflow to 0
log_sum = np.sum(np.log(probs))  # Safe: ≈ -0.50
print("Direct product:", product, "Log sum:", log_sum)
```

Softmax — Scores to Probabilities

Converts arbitrary scores to probability distribution

Definition

- $\text{softmax}(\text{scores})_i = \exp(\text{score}_i) / \Sigma\exp(\text{score}_j)$
- Outputs sum to 1 (valid probability)
- Larger scores → larger probabilities

ML Role

- Multi-class classification output layer
- Attention mechanisms
- Preserves ranking while normalizing

NAIVE SOFTMAX

```
def softmax_naive(scores):
    exp_scores = np.exp(scores)
    return exp_scores / np.sum(exp_scores)
scores = np.array([1.0, 2.0, 3.0])
print("Probs:", softmax_naive(scores), "Sum:", softmax_naive(scores).sum())
```

Numerical Instability in Naive Softmax

Large scores cause overflow

The Problem

- exp(1000) overflows to inf
- Division by inf produces NaN
- Training breaks silently with NaN loss!

Symptom

- NaN in loss or gradients
- Model produces inf predictions
- Common in early training with bad init

OVERFLOW EXAMPLE

```
big_scores = np.array([1000.0, 1001.0, 1002.0])
print("exp(1000):", np.exp(1000))  # inf!
print("Naive softmax:", softmax_naive(big_scores))  # [nan, nan, nan]
```

Numerically Stable Softmax

Subtract max before exponentiating

The Trick

- Subtract max(scores) from all scores
- Mathematically equivalent (cancels in ratio)
- Largest exponent becomes exp(0) = 1

Why It Works

- All exponents now ≤ 0
- No overflow possible
- Used in all ML frameworks

STABLE SOFTMAX

```
def softmax_stable(scores):
    shifted = scores - np.max(scores)  # Max becomes 0
    exp_scores = np.exp(shifted)        # All exp ≤ 1
    return exp_scores / np.sum(exp_scores)
print("Stable:", softmax_stable(big_scores))  # Works! [0.09, 0.24, 0.67]
```

Log-Softmax — Directly in Log Space

Combines softmax and log efficiently

Motivation

- Cross-entropy uses log(softmax(scores))
- Computing separately wastes computation
- Log-softmax is more stable

Formula

- $\text{log_softmax}(x)_i = x_i - \max - \log(\Sigma\exp(x_j - \max))$
- Frameworks: log_softmax + NLLLoss
- Avoids extra exp/log roundtrip

LOG-SOFTMAX

```
def log_softmax_stable(scores):
    shifted = scores - np.max(scores)
    return shifted - np.log(np.sum(np.exp(shifted)))
log_probs = log_softmax_stable(np.array([2, 1, 0.1]))
print("Log probs:", log_probs, "\nSum probs:", np.exp(log_probs).sum())
```

Softmax vs Argmax

Probabilities vs hard decisions

Argmax

- Returns index of largest value
- Hard decision, discards uncertainty
- Used at inference for final prediction

Softmax

- Returns full probability distribution
- Retains uncertainty information
- Needed for calibration and risk analysis

SOFTMAX VS ARGMAX

```
scores = np.array([1.0, 2.5, 0.3])
probs = softmax_stable(scores)
pred = np.argmax(probs)  # Hard decision: class 1
print("Probs:", probs, "Predicted class:", pred)
```

Temperature Scaling

Control sharpness of distribution

Temperature T

- softmax(scores / T)
- T < 1: sharper (more confident)
- T > 1: smoother (less confident)

ML Uses

- Model calibration (calibrate confidence)
- Attention sharpness control
- Knowledge distillation (soften teacher)

TEMPERATURE SCALING

```
def softmax_temp(scores, T=1.0):
    return softmax_stable(scores / T)
for T in [0.5, 1.0, 2.0]:
    print(f"T={T}:", softmax_temp(np.array([2, 1, 0]), T))
```

Log-Sum-Exp Trick

Stable computation of $\log(\Sigma\exp(a_i))$

The Pattern

- $\log(\Sigma\exp(a_i))$ overflows if a_i large
- Solution: factor out max(a)
- $LSE(a) = \max(a) + \log(\Sigma\exp(a_i - \max(a)))$

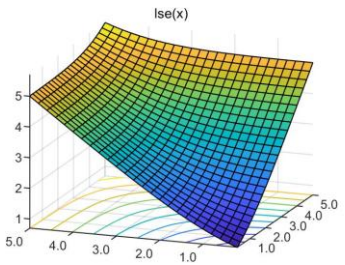
Applications

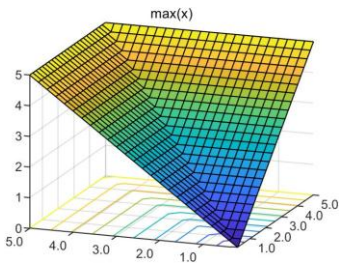
- Log-softmax denominator
- CRF partition functions
- Any probabilistic model in log space

LOG-SUM-EXP

```
def log_sum_exp(a):
    a_max = np.max(a)
    return a_max + np.log(np.sum(np.exp(a - a_max)))
vals = np.array([1000, 1001, 1002])
print("Stable LSE:", log_sum_exp(vals))  # ≈ 1002.4
```

$$\mathrm{LogSumExp}(x_1 \ldots x_n) = \log\left(\sum_{i=1}^{n} e^{x_i}\right)$$





DAY 44 · ACTIVITY

Softmax Implementation

Build and test stable softmax

Activity Steps

1. Implement naive and stable softmax
2. Test with small and large scores
3. Verify probabilities sum to 1
4. Check stable handles [1000, 1001, 1002]
5. Implement log_softmax
6. Compare to np.log(softmax(scores))

Assessment Questions

Q1: Why use log probabilities in ML?

Q2: Implement stable softmax, explain max trick?

Q3: How does softmax relate to confidence?

Q4: When does naive softmax break training?

Q5: Compute cross-entropy given probs?

DAY 45

Math Mini-Project

Integrating Linear Algebra + Probability

OBJECTIVES

- Use vectors/matrices for data and parameters
- Use distributions to simulate data/noise
- Apply softmax and probability calculations
- Practice explaining math in plain language
- Build complete synthetic classification

ACTIVITY

Build a notebook demonstrating key math concepts in ML task.

ASSESSMENT

Graded submission with code, shapes, interpretations.

DAY 45 · MINI-PROJECT

Synthetic Classification Task Setup

Simulate 3-class data from multivariate normals

Task

- 3 classes from different normal distributions
- Each class has different mean vector
- Same covariance matrix for all classes

Why This Setup

- Shows how distributions generate data
- Realistic classification scenario
- Tests all learned math concepts

DATA GENERATION

```
n = 200; cov = np.array([[1, 0.2], [0.2, 1]])
X1 = np.random.multivariate_normal([0, 0], cov, n)
X2 = np.random.multivariate_normal([3, 0], cov, n)
X3 = np.random.multivariate_normal([0, 3], cov, n)
X = np.vstack([X1, X2, X3]); y = np.array([0]*n + [1]*n + [2]*n)
```

DAY 45 · MINI-PROJECT

Linear Classifier with Softmax

W, b, and softmax predictions

Model Setup

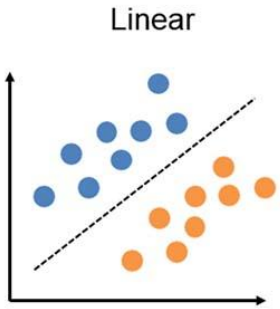
- W: (n_features, n_classes) weight matrix
- b: (n_classes,) bias vector
- scores = X @ W + b, probs = softmax(scores)

Output Interpretation

- Each row of probs: distribution over 3 classes
- Probabilities sum to 1 per sample
- Random W gives ~random predictions

LINEAR CLASSIFIER

```
W = np.random.randn(2, 3) * 0.5; b = np.zeros(3)
def softmax_batch(s): shifted = s - s.max(1, keepdims=True); e = np.exp(shifted); return e / e.sum(1,
keepdims=True)
scores = X @ W + b; probs = softmax_batch(scores)
```



Predictions and Accuracy

argmax for class, mean for accuracy

Prediction Rule

- Predicted class = argmax of probabilities
- For each sample, pick highest-prob class
- Accuracy = fraction correct

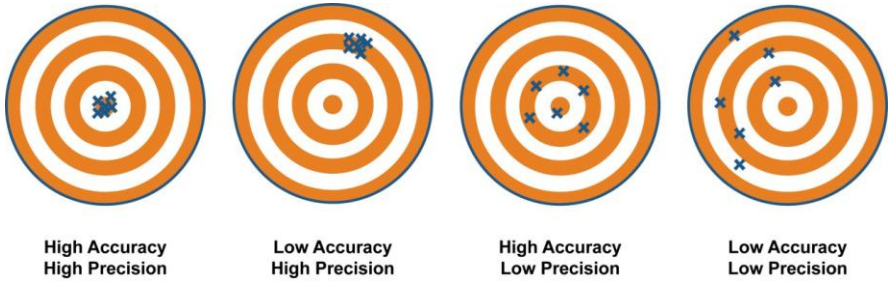
Interpretation

- Accuracy is empirical probability of success
- Random W → ~33% accuracy (chance)
- Need training to improve!

ACCURACY CALCULATION

```
y_pred = np.argmax(probs, axis=1)
accuracy = np.mean(y_pred == y)
print("Predicted labels (first 10):", y_pred[:10])
print("Initial accuracy:", accuracy)  # ~33% with random W
```

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$



Cross-Entropy Loss

-mean(log(p_true_class))

Definition

- For each sample: -log(prob of true class)
- Average over all samples
- Lower = better predictions

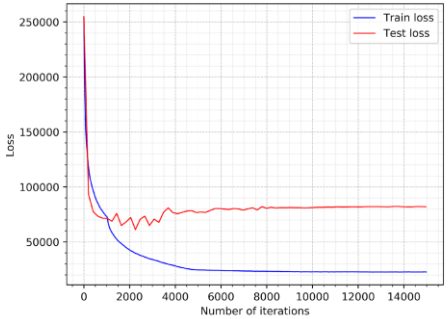
ML Connection

- Negative log-likelihood objective
- Connects softmax to optimization
- Foundation of deep learning classification

CROSS-ENTROPY LOSS

```
def cross_entropy(probs, y_true):
    n = len(y_true)
    p_true = probs[np.arange(n), y_true]  # Gather true class probs
    return -np.mean(np.log(p_true + 1e-12))  # Add epsilon for stability
print("Initial loss:", cross_entropy(probs, y))
```

$$L = -\sum_{k=1}^{K} y_k \log(p_k)$$



DAY 45 · MINI-PROJECT

Probability-Based Confidence Analysis

High vs low confidence predictions

Analysis

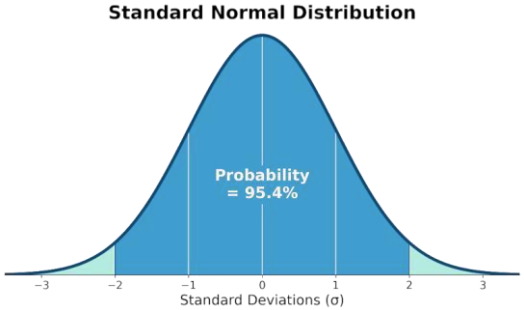
- max_prob > 0.9: high confidence
- max_prob < 0.5: ambiguous/uncertain
- Fractions give insight into model behavior

ML Implications

- Informs thresholding decisions
- Identifies samples for human review
- Guides active learning strategies

CONFIDENCE ANALYSIS

```
max_probs = probs.max(axis=1)
print("High confidence (>0.9):", np.mean(max_probs > 0.9))
print("Low confidence (<0.5):", np.mean(max_probs < 0.5))
```



DAY 45 · MINI-PROJECT

Linear Algebra for Feature Transforms

X_new = X @ A transforms features

Concept

- Linear transformation rotates/scales data
- Feature distributions change
- Many layers are just linear transforms + nonlinearity

Neural Network View

- Dense layer: linear transform + activation
- Learned transforms improve separation
- This project shows the math behind it

FEATURE TRANSFORMATION

```
A = np.array([[1, 0.5], [-0.2, 0.8]])  # 2x2 transform
X_transformed = X @ A
print("Original X mean:", X.mean(axis=0))
print("Transformed mean:", X_transformed.mean(axis=0))
```

DAY 45 · ACTIVITY

Build Math-For-ML Notebook

Integrate all concepts

Notebook Sections

1. Linear Algebra: vectors, dot products, matrices
2. Distributions: simulation and visualization
3. Probabilities: PDF, CDF, softmax
4. Mini-classifier: synthetic data, predictions
5. Analysis: accuracy, loss, confidence
6. Explanation: interpret each operation

Grading Criteria

- Correctness: code runs, uses NumPy/SciPy
- Coverage: LA, probability, distributions, softmax
- Interpretation: bullets explain each result
- Clarity: clear names, shape annotations
- Insight: connects math to ML behavior

Week 9 Complete!

Math for ML II — Probability and Core Functions

Probability	Long-run frequency, events, empirical vs theoretical, Monte Carlo, conditional
Distributions	Uniform, normal, mean/variance, skewness, multimodality, CLT
Computing Probs	PDF, CDF, ppf with SciPy; tail probabilities; Monte Carlo validation
Log/Exp/Softmax	Numerical stability, log-sum-exp trick, temperature scaling, cross-entropy
Mini-Project	Synthetic classification integrating LA + probability + softmax

Key Takeaway

Probability and numerical stability are essential for ML. Understanding distributions, computing probabilities, and implementing stable softmax prepares you for classification, calibration, and deep learning.

THE ML PROBABILITY CHAIN

```
probs = softmax_stable(X @ W + b)  # Scores → probabilities
loss = -np.mean(np.log(probs[range(n), y]))  # Cross-entropy loss
```